

作业二

1. 运行汇编代码

1.1 更改指令编写地址

原指令在编译后的内存空间如下

```
C:0x003B 00 NOP
C:0x003C 00 NOP
C:0x003D 00 NOP
C:0x003E 00 NOP
C:0x003F 00 NOP
5: START: MOV SP, #05H
758105 MOV SP(0x81), #0x05
6: LOOP: NOP
C:0x0043 00 NOP
7: LJMP LOOP
```

```
1  ORG 0000H
2  LJMP START
3
4  ORG 0040H
5  START: MOV SP, #05H
6  LOOP:  NOP
7  LJMP LOOP
8
9  END
```

可以发现 START 标号开始于内存的 0x0040

在更改为 ORG 0100H 后，得到下图

```
C:0x00FE 00 NOP
C:0x00FF 00 NOP
5: START: MOV SP, #05H
758105 MOV SP(0x81), #0x05
6: LOOP: NOP
C:0x0103 00 NOP
7: LJMP LOOP
C:0x0104 020103 LJMP LOOP(0:0103)
C:0x0107 00 NOP
```

```
1  ORG 0000H
2  LJMP START
3
4  ORG 0100H
5  START: MOV SP, #05H
6  LOOP:  NOP
7  LJMP LOOP
8
9  END
```

发现 START 标号开始地址变为 0x0100H，证明了 ORG 的作用是指定下面代码的开始地址

1.2 所有指令的机器码

```
ORG    0000H    ;伪指令，无机器码

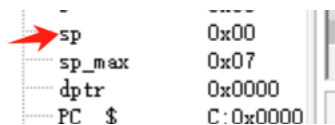
LJMP START      ;02 1234，很显然，02 是 LJMP 的机器码
                ;而内存中立即数保存为 1234，是 START 标号的位置
                ;说明 51 是大端存储，这也使得入栈后 sp 向上涨变得合理

ORG 1234H      ;伪指令，无机器码
START: MOV SP, #34H ;75 81 34，75 是 MOV 的机器码，81 是寄存器 SP 的编号
                ;05 是立即数的编码
LOOP:  NOP      ;00 是 NOP 的机器码
      LJMP LOOP ;02 1237，同上一条 LJMP，1237 是 LOOP 标号的位置

END           ;伪指令，无机器码
```

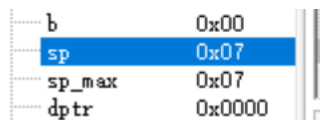
1.3 SP 的变化

当程序没有运行时，SP = 00H



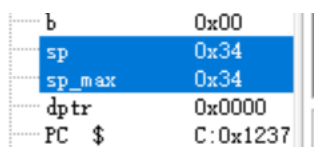
sp	0x00
sp_max	0x07
dptr	0x0000
PC \$	C:0x0000

当程序开始运行时，SP 初始化为 07H



b	0x00
sp	0x07
sp_max	0x07
dptr	0x0000

当运行完 MOV 指令后，SP 变为 34H（我的程序改了一些）



b	0x00
sp	0x34
sp_max	0x34
dptr	0x0000
PC \$	C:0x1237

同时，发现下方的 SP_max 变量一直在记录 SP 寄存器的最大值，推测是用来记录栈空间，便于发现是否栈溢出

1.4 PC 的变化轨迹

上电复位后, PC=0000H

```
dptr      0x0000
PC $      C:0x0000
states    1
```

运行完 LMP START 后, PC=1234H

```
dptr      0x0000
PC $      C:0x1234
states    2
```

运行完 MOV SP, #34H 后, PC=1237H

```
dptr      0x0000
PC $      C:0x1237
states    3
```

运行完 NOP 后, PC=1238H

```
dptr      0x0000
PC $      C:0x1238
states    4
```

运行完 LMP LOOP 后, PC=1237H, 回到了 LOOP 标号处

```
dptr      0x0000
PC $      C:0x1237
states    5
```

然后重复最后两步, 因为这是个死循环

1.5 标号的含义

仅作为内存地址的标记, 在程序第二次编译后就变成了具体的地址
LMP 作为绝对跳转指令, 其机器码也是具体的地址而非偏移量

2. PSW 的用处

测试程序如下

```
ORG 0000H
```

```
LJMP START
```

```
ORG 1234H
```

```
START: MOV SP, #34H
```

```
; Test Case 1: Addition (No Carry)
```

```
MOV A, #12H
```

```
ADD A, #34H
```

```
; Test Case 2: Addition (With Carry)
```

```
MOV A, #0F0H
```

```
ADD A, #10H
```

```
; Test Case 1: Subtraction (No Borrow)
```

```
MOV A, #50H
```

```
SUBB A, #30H
```

```
; Test Case 2: Subtraction (With Borrow)
```

```
MOV A, #10H
```

```
SUBB A, #30H
```

```
; Test Case 1: Multiplication (Result fits in 8 bits)
```

```
MOV A, #08H
```

```
MOV B, #02H
```

```
MUL AB
```

```
; Test Case 2: Multiplication (Result exceeds 8 bits)
```

```
MOV A, #0FFH
```

```
MOV B, #02H
```

```
MUL AB
```

```
; Test Case 1: Division (Exact Division)
```

```
MOV A, #10H
```

```
MOV B, #02H
```

```
DIV AB
```

```
; Test Case 2: Division 0
```

```
MOV A, #11H
```

```
MOV B, #00H
```

DIV AB

LOOP: NOP

LMP LOOP

END

PSW 的定义如下

PSW位地址	D7H	D6H	D5H	D4H	D3H	D2H	D1H	D0H
	CY	AC	F0	RS1	RS0	OV		P

每一个 case 的测试结果如下

1. 12H+34H = 0001 0010 + 0011 0100 = 0100 0110, 预期结果只有 P=1

states

0.00000300

psw

0x01

p

1

f1

0

ov

0

rs

0

f0

0

ac

0

cy

0

7

8

9

10

11

12

13

14

15

; Test Case 1: Addition (No Carry)

MOV A, #12H

ADD A, #34H

; Test Case 2: Addition (With Carry)

MOV A, #0F0H

ADD A, #10H

; Test Case 3: Subtraction (No Borrow)

实际结果与预测相同

2. F0H+10H = 1111 0000 + 0001 0000 = 1 0000 0000, 会产生一个溢出

sec

0.00000400

psw

0x80

p

0

f1

0

ov

0

rs

0

f0

0

ac

0

cy

1

7

8

9

10

11

12

13

14

15

16

; Test Case 1: Addition (No Carry)

MOV A, #12H

ADD A, #34H

; Test Case 2: Addition (With Carry)

MOV A, #0F0H

ADD A, #10H

; Test Case 3: Subtraction (No Borrow)

MOV A, #50H

实际结果与预测相同, 后续不再赘述

3. 50H-30H = 0101 0000 + 1101 0000 = 0010 0000

sec

0.00000500

psw

0x41

p

1

f1

0

ov

0

rs

0

f0

0

ac

1

cy

0

12

13

14

15

16

17

18

19

20

21

22

MOV A, #0F0H

ADD A, #10H

; Test Case 1: Subtraction (No Borrow)

MOV A, #50H

SUBB A, #30H

; Test Case 2: Subtraction (With Borrow)

MOV A, #10H

SUBB A, #30H

为什么 ac=1?

4. $10H - 30H = 0001\ 0000 + 1101\ 0000 = 1110\ 0000$, 不够减的变负了, 最高位借 1

states	12
sec	0.00000600
psw	0x81
p	1
f1	0
ov	0
rs	0
f0	0
ac	0
cy	1

```

15 ; Test Case 1: Subtraction (No Borrow)
16 MOV A, #50H
17 SUBB A, #30H
18
19 ; Test Case 2: Subtraction (With Borrow)
20 MOV A, #10H
21 SUBB A, #30H
22
23 ; Test Case 1: Multiplication (Result fits in 8 bits)
24 MOV A, #08H
25 MOV B, #02H

```

5. $08H * 02H = 8 * 2 = 16 = 0001\ 0000$

states	12
sec	0.00000950
psw	0x01
p	1
f1	0
ov	0
rs	0
f0	0
ac	0
cy	0

```

21 SUBB A, #30H
22
23 ; Test Case 1: Multiplication (Result fits in 8 bits)
24 MOV A, #08H
25 MOV B, #02H
26 MUL AB
27
28 ; Test Case 2: Multiplication (Result exceeds 8 bits)
29 MOV A, #0FFH

```

6. $0FFH * 02H = 255 * 2 = 1\ 1111\ 1110$, 溢出了

states	12
sec	0.00001300
psw	0x05
p	1
f1	0
ov	1
rs	0
f0	0
ac	0
cy	0

```

28 ; Test Case 2: Multiplication (Result exceeds 8 bits)
29 MOV A, #0FFH
30 MOV B, #02H
31 MUL AB
32
33 ; Test Case 1: Division (Exact Division)
34 MOV A, #10H
35 MOV B, #02H
36 DIV AB

```

7. $10H / 02H = 0000\ 1000$

states	12
psw	0x01
p	1
f1	0
ov	0
rs	0
f0	0
ac	0
cy	0

```

29 MOV A, #0FFH
30 MOV B, #02H
31 MUL AB
32
33 ; Test Case 1: Division (Exact Division)
34 MOV A, #10H
35 MOV B, #02H
36 DIV AB

```

8. $11H / 00H$, 溢出

states	12
sec	0.00002000
psw	0x04
p	0
f1	0
ov	1
rs	0
f0	0
ac	0
cy	0

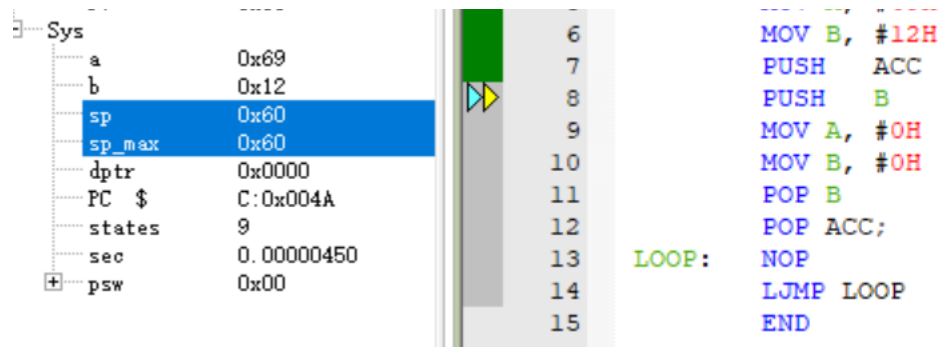
```

28 ; Test Case 2: Multiplication (Result exceeds 8 bits)
29 MOV A, #0FFH
30 MOV B, #02H
31 MUL AB
32
33 ; Test Case 1: Division (Exact Division)
34 MOV A, #10H
35 MOV B, #02H
36 DIV AB
37
38 ; Test Case 2: Division 0
39 MOV A, #11H
40 MOV B, #00H
41 DIV AB

```

3. 观察栈操作

上电后 SP=07H，赋值后 SP=5FH，这些略去
在执行 PUSH ACC 后，得到下图结果

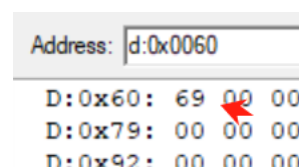


Register Window:

Register	Value
a	0x69
b	0x12
sp	0x60
sp_max	0x60
dptr	0x0000
PC	0x004A
states	9
sec	0.00000450
psw	0x00

Assembly Code:

```
MOV B, #12H
PUSH ACC
PUSH B
MOV A, #0H
MOV B, #0H
POP B
POP ACC;
LOOP: NOP
LJMP LOOP
END
```



Memory Window:

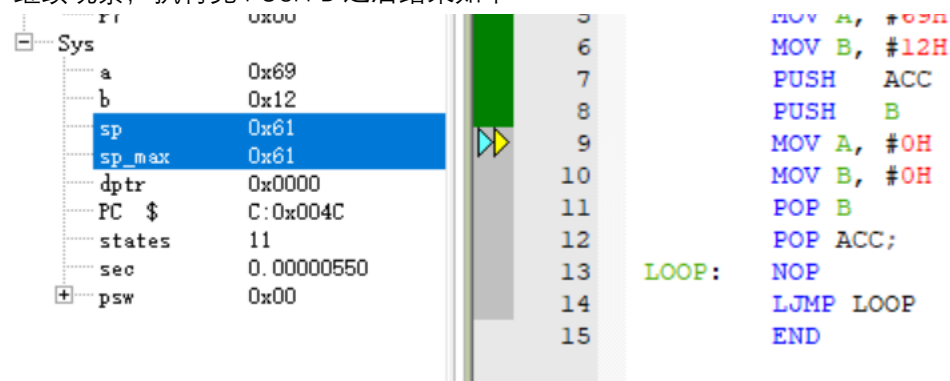
Address: 0x0060

Address	Value
0x0060	69 00 00
0x0079	00 00 00
0x0092	00 00 00

发现 SP += 1，数据内存内 0060H 处的值变成了 ACC 的值，也就是 PUSH 的过程是

1. SP += 1
2. (SP) = ACC

继续观察，执行完 PUSH B 之后结果如下

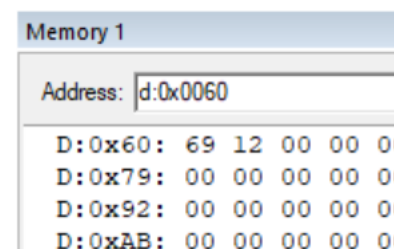


Register Window:

Register	Value
a	0x69
b	0x12
sp	0x61
sp_max	0x61
dptr	0x0000
PC	0x004C
states	11
sec	0.00000550
psw	0x00

Assembly Code:

```
MOV A, #09H
MOV B, #12H
PUSH ACC
PUSH B
MOV A, #0H
MOV B, #0H
POP B
POP ACC;
LOOP: NOP
LJMP LOOP
END
```



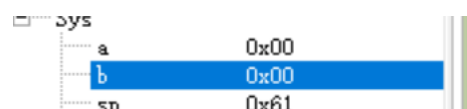
Memory Window:

Address: 0x0060

Address	Value
0x0060	69 12 00 00 00
0x0079	00 00 00 00 00
0x0092	00 00 00 00 00
0x00AB	00 00 00 00 00

同样的，SP += 1，内存地址 0061H 处值变为 B 的值

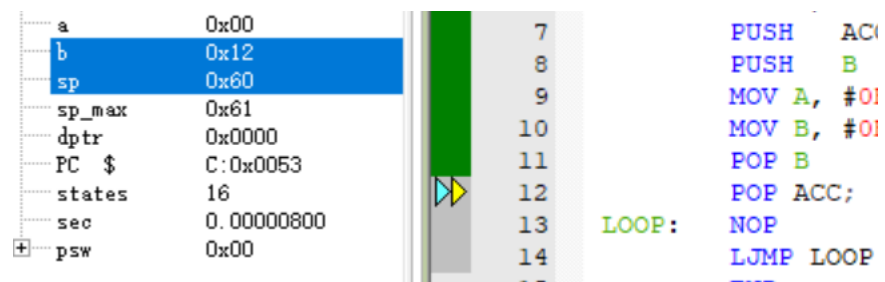
然后执行对 ACC，B 的赋值操作，得到清空后的寄存器



Register Window:

Register	Value
a	0x00
b	0x00
sp	0x61

执行 POP B 指令

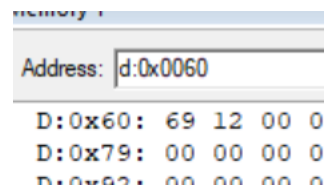


a	0x00	7
b	0x12	8
sp	0x60	9
sp_max	0x61	10
dptr	0x0000	11
PC \$	C:0x0053	12
states	16	13
sec	0.00000800	14
psw	0x00	15

```

PUSH ACC
PUSH B
MOV A, #01
MOV B, #01
POP B
POP ACC;
LOOP: NOP
LJMP LOOP

```



Address: d:0x0060

D:0x60: 69 12 00 00

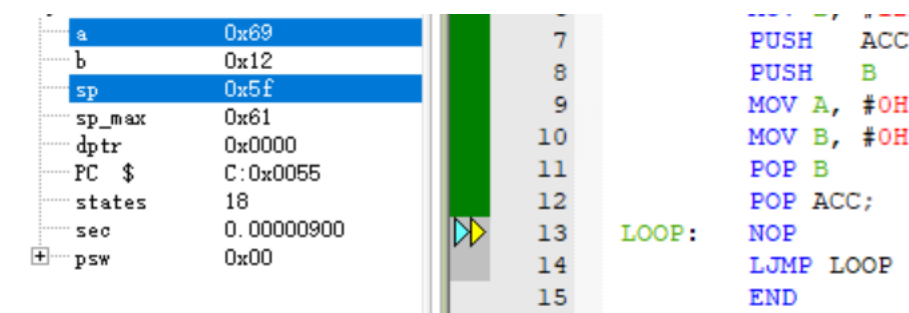
D:0x79: 00 00 00 00

D:0x92: 00 00 00 00

可以发现 B 被赋值为了内存 0061H 处的值，然后 SP -= 1，0061H 处的值并不会清空
因此，POP 指令依次实现

1. B = (SP)
2. SP -= 1

再执行 POP ACC，得到类似的结果，不赘述

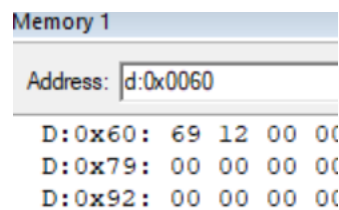


a	0x69	7
b	0x12	8
sp	0x5f	9
sp_max	0x61	10
dptr	0x0000	11
PC \$	C:0x0055	12
states	18	13
sec	0.00000900	14
psw	0x00	15

```

PUSH ACC
PUSH B
MOV A, #0H
MOV B, #0H
POP B
POP ACC;
LOOP: NOP
LJMP LOOP
END

```



Address: d:0x0060

D:0x60: 69 12 00 00

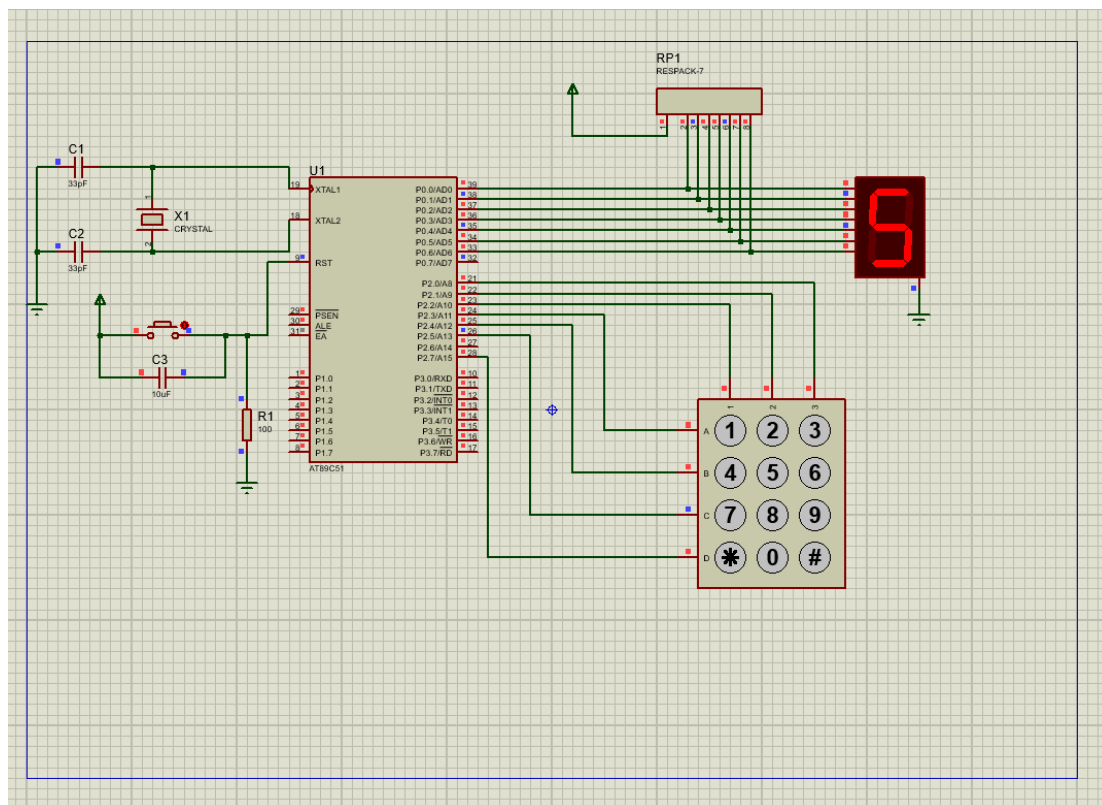
D:0x79: 00 00 00 00

D:0x92: 00 00 00 00

由此得出，栈是一种先进后出的数据结果（Fist in, last out, FILO）

4. 复现七段数码管实验

最终复现结果如下，可以跟随按键进行显示



硬件方面，分为两个大块

- 左边的部分用于组成 8051 的最小系统
- 右边的部分用于输入数字和显示数字

软件方面，也是两块：

```
TEST:      MOV P2, #0FFH
           CLR P2.3
           JNB P2.2, NUM1
           JNB P2.1, NUM2
           JNB P2.0, NUM3
           SETB P2.3
           CLR P2.4
           JNB P2.2, NUM4
           JNB P2.1, NUM5
           JNB P2.0, NUM6
           SETB P2.4
           CLR P2.5
           JNB P2.2, NUM7
           JNB P2.1, NUM8
           JNB P2.0, NUM9
           SETB P2.5
           CLR P2.6
           JNB P2.1, NUM0
           SETB P2.6
           SJMP TEST
```

这段程序的每一个循环如下：

首先给所有引脚加上高电平，然后按顺序依次清空、设置引脚 3，4，5，6 的电压，以 3 号脚为例：清空后 3 号脚低电平，此时若按下按钮上的 1，那么 3 号脚和 2 号脚导通，2 号脚输入低电平，触发 NUM1

其他的分支语句也都大差不差。最后当 3 号角对于的几个按钮都检查完成后，重新置为高电平，以免影响其他引脚检测。

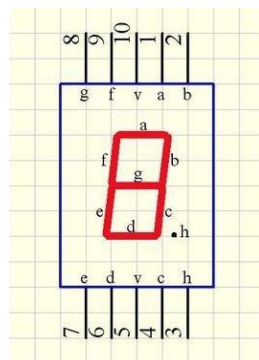
整个模块结束后有一个跳转，保证了程序能够持续检测这些引脚

```

NUM0:    MOV P0,#3FH
          SJMP TEST
NUM1:    MOV P0,#06H
          SJMP TEST
NUM2:    MOV P0,#5BH
          SJMP TEST
NUM3:    MOV P0,#4FH
          SJMP TEST
NUM4:    MOV P0,#66H
          SJMP TEST
NUM5:    MOV P0,#6DH
          SJMP TEST
NUM6:    MOV P0,#7DH
          SJMP TEST
NUM7:    MOV P0,#07H
          SJMP TEST
NUM8:    MOV P0,#7FH
          SJMP TEST
NUM9:    MOV P0,#6FH
          SJMP TEST
END

```

这些语句是通过前一个模块的识别来执行的。以 NUM1 为例，当按钮的 1 被按下时，2 号口变为低电平，跳转到 NUM1，是为了在七段数码管输出 1 的形状的代码。根据七段数码管的定义，当显示为 1 时，对应的 7 个管分别为：b,c=1,其他=0，那么根据管脚的定義，P0 口需要輸出 0000 0110B = 06H，最高位置零（悬空的）



显示完成后返回 TEST 进行持续的检测。其他数字同理，不赘述